

Hierarchical Navigable Small World Graphs (HNSW)

Efficient Approximate Nearest Neighbor Search

Presentation Outline

- 1- Overview of the Paper
- 2- K-NNS VS K-ANNS
- 3- NSW, Hierarchy and connection to skip list
- 4- Related work and their Problems
- 5-How HNSW Fixes These Problems
- 6- HNSW algorithms
- 7- Effect of Parameters (mL , M , ef)
- 8- Complex Analysis
- 9- Comparison to State-of-Art

Overview

The paper proposes a way to search in huge datasets using a graph built in layers. The upper layers help you jump quickly to the right place, and the lower layers help you find the exact answer. Each point appears in some layers randomly, so higher layers are smaller. The search starts from the top and goes down until it finds the nearest neighbors. This makes the search much faster and more accurate than older methods.

K-NNS VS K-ANNS

Why we used K-ANNS instead of K-NNS?

What is K-NNS?

- K-Nearest Neighbor Search (K-NNS) finds the exact K closest points to a query.
- Guarantees 100% correct result
- A naive approach to K-NNS is to compute the distances between the query and every element in the dataset and select the elements with minimal distance
- So, it is not infeasible for large datasets

What is K-ANNS?

- K-Approximate Nearest Neighbor Search (K-ANNS) allows a very small error in the result.
- it gives huge speed improvements.
- This makes K-ANNS the practical choice for large-scale systems.

NSW (Navigable Small World)

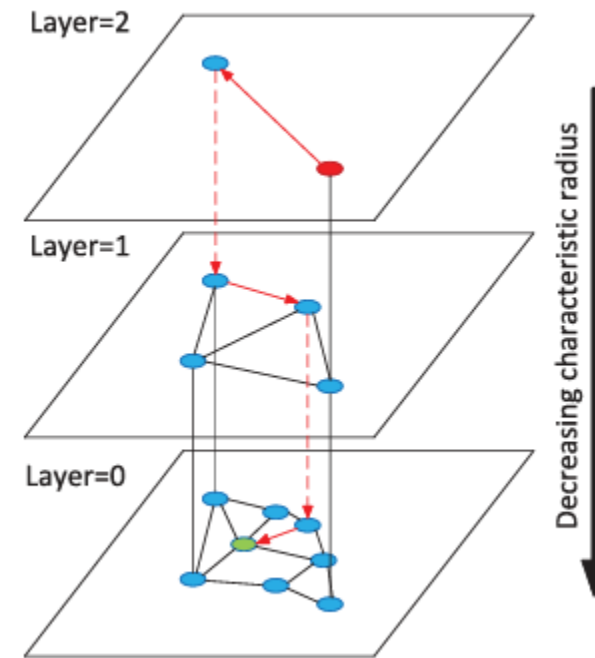
- NSW is Graph-based for Approximate Nearest Neighbor (ANN) search.
- Each data point is a **node** in a graph.
- Each node is connected to a **small set of nearby nodes**.
- Search is done by **greedy navigation**:
 - Start from an entry point.
 - Move to a neighbor that is closer to the query.
 - Repeat until no improvement is possible.

Limitations of NSW:

- Some nodes become hubs with many connections.
(e.g. normal node have 10 neighbor ---> so we will check 10 neighbor only, but Hub node have 500 neighbor ---> so we will check 500)
- Search cost **grows with dataset size**.
- Performance becomes **unstable on clustered data**.

Why Making NSW Hierarchical? (HNSW)

- Instead of using **one big graph**, HNSW builds **multiple layers of graphs**.
- **Upper layers:**
 - Contain **few nodes**
 - Provide **long-range jumps**
- **Lower layers:**
 - Contain **many nodes**
 - Provide **accurate search**
- **Search:**
 - Starts at the **top layer**
 - Goes **down layer by layer**
- This makes the search:
- **Faster**
- **More scalable**
- **More stable**



Connection to Skip List

- Hierarchical NSW algorithm can be seen as an extension of skip list structure.

Skip List:

- Uses **multiple levels** of linked lists.
- Upper levels act as **shortcuts**.

HNSW:

- Uses **multiple levels** of **proximity graphs**.
- Upper levels also act as **shortcuts**.

Related Work there Problems

-Proximity Graph Methods:

They build a graph where each point is connected to its nearest neighbors, and search by moving to closer and closer points.

Main problem:

- When the dataset becomes large, the search becomes **slow**.
- If the data is **clustered**, the search can get **stuck in the cluster**.

-Navigable Small World (NSW):

It improves the graph by adding some long connections to jump faster across the data.

Main problem:

- When the dataset grows, the search is **still not fast enough**.
- The performance is **not always stable**.

How HNSW Fixes These Problems

(1)

HNSW does not use one big graph.
Instead, it builds **many layers of graphs**.

Top layers:

- very small
- help you jump quickly to the right area

Bottom layers:

- very large
- used to find the final accurate result

Search:

- Starts from the top then goes down layer by layer

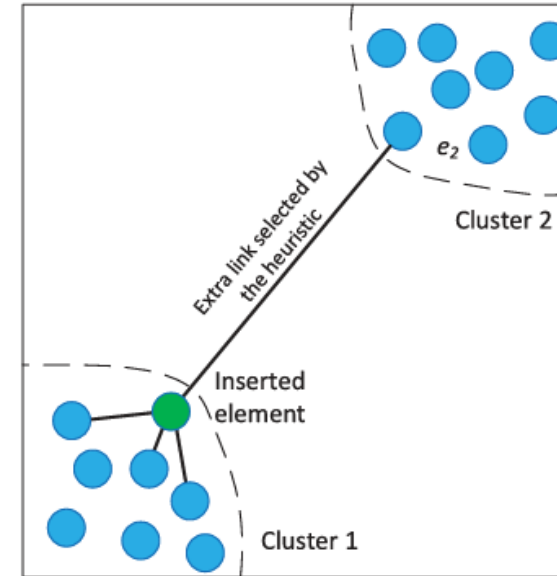
How HNSW Fixes These Problems

(2)

It builds the graph in a smart way:

- It chooses neighbors **carefully** when adding new points.
- This keeps the graph **well connected**.

- So the search does not get stuck inside one cluster.



When a new point is inserted near the boundary of two clusters, selecting only the closest neighbors would connect it to one cluster only. The heuristic adds an extra link to the other cluster to keep the graph well connected and prevent the search from getting stuck.

HNSW: Algorithm Overview

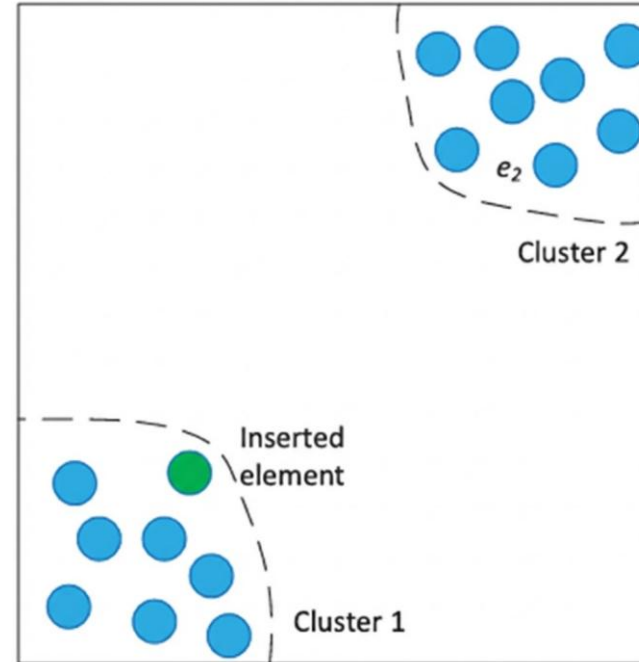
- HNSW has two main operations:

1- Insertion

2- Search

1- Insert a new element

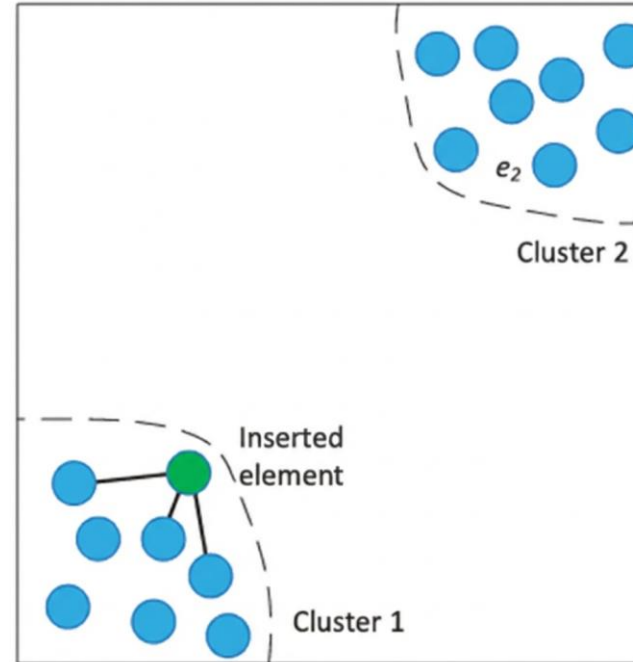
We insert a new point into the graph



1- Insert a new element

Connect to nearest neighbors.

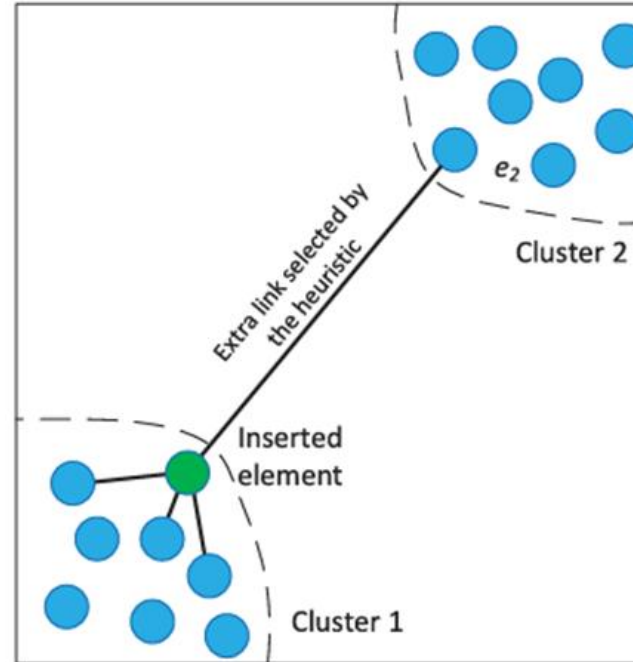
the new point is connected to its nearest neighbors in the same cluster



1- Insert a new element

Add extra link using heuristic.

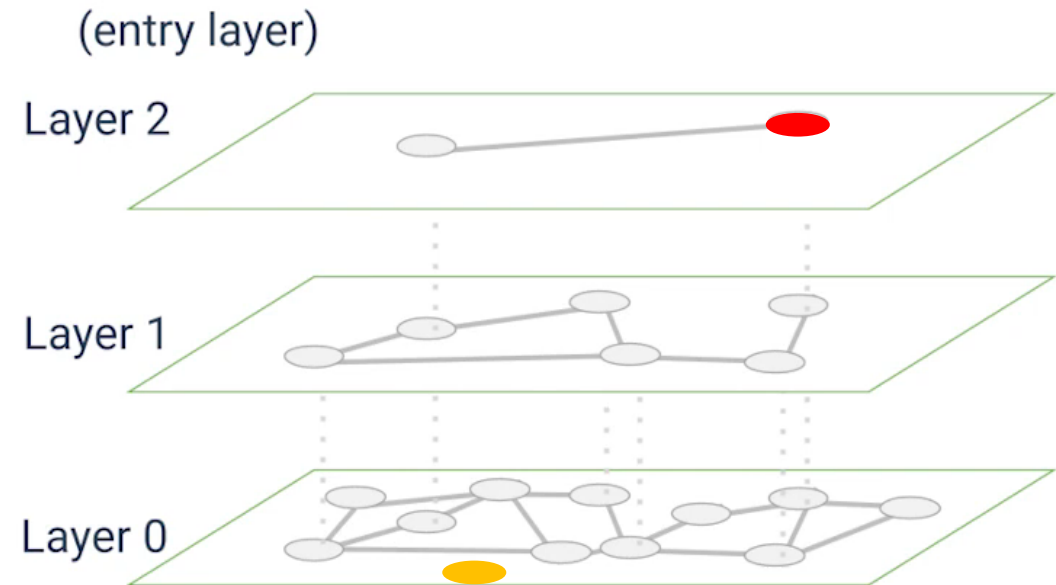
HNSW adds an extra link to another cluster to keep the graph well connected



2- Search

Step 1: Entry Point at the Top Layer

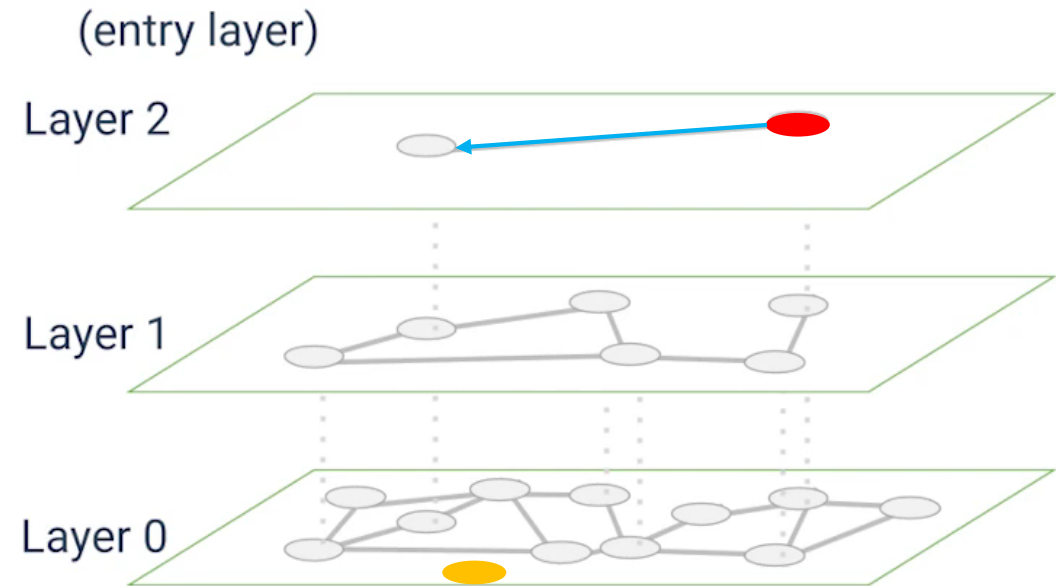
- search for a given query (**yellow dot**)
- Start from an **random entry point (red dot)** in the **top layer**.
- The top layer is **very small**.
- It allows **fast global navigation**.



2- Search

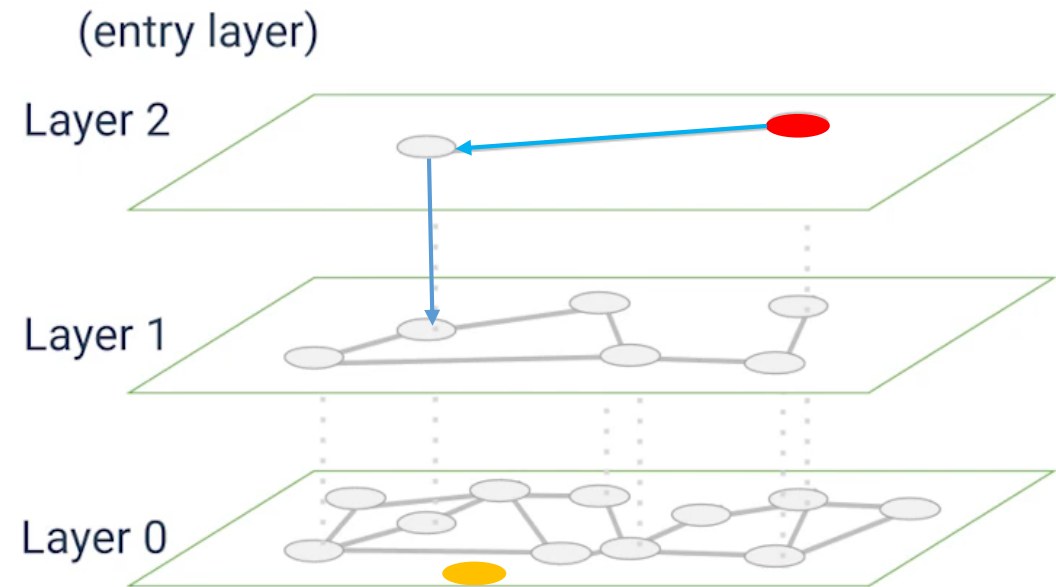
Step 2: Find the local minimum in the current layer

- we perform a **greedy search** until we find the local minimum



2- Search

Step 3: Go down a layer



2- Search

We repeat this process until we reach the bottom layer and find the nearest neighbor (**green dot**) to the query

